Hadoop & Spark - ISEN 2025/2026

Lab: Spark Tables

For this Lab you will need:

- The 3.8 Lab

As a reminder

Check 3.1 to login in NGROK, Kaggle, install Java or Spark and import libraries.

Spark Tables

For this part we will need a new Spark Session with Hive in order to be on a persistent catalog and not an in-memory one (default):

```
spark = (
    SparkSession
      .builder
      .appName("CollabTableApp")
      .master("local[*]")
      .config('spark.ui.port', '4050')
      .config("spark.dynamicAllocation.enabled", "false")
      .config("spark.sql.adaptive.enabled", "false")
      .enableHiveSupport() # Enable Hive support
      .getOrCreate()
)
spark.conf.set("spark.sql.repl.eagerEval.enabled", True)

sc = spark.sparkContext

spark
```

Spark Tables

If at this point you lost your UI link is here how to recover it:

```
get_ipython().system_raw('./ngrok http 4050 &')
!curl -s http://localhost:4040/api/tunnels | grep -Po 'public_url":"(?=https)\K[^"]*'
```

Spark Tables

Once again make sure the yellow Taxis are loaded:

```
yellowTaxiSchema = (
                        StructType
                        ([
                            StructField("VendorId"                , IntegerType()   , True),
                            StructField("lpep_pickup_datetime"    , TimestampType() , True),
                            StructField("lpep_dropoff_datetime"   , TimestampType() , True),
                            StructField("passenger_count"         , DoubleType()    , True),
                            StructField("trip_distance"           , DoubleType()    , True),
                            StructField("RatecodeID"              , DoubleType()    , True),
                            StructField("store_and_fwd_flag"      , StringType()    , True),
                            StructField("PULocationID"            , IntegerType()   , True),
                            StructField("DOLocationID"            , IntegerType()   , True),
                            StructField("payment_type"            , IntegerType()   , True),
                            StructField("fare_amount"             , DoubleType()    , True),
                            StructField("extra"                   , DoubleType()    , True),
                            StructField("mta_tax"                 , DoubleType()    , True),
                            StructField("tip_amount"              , DoubleType()    , True),
                            StructField("tolls_amount"            , DoubleType()    , True),
                            StructField("improvement_surcharge"   , DoubleType()    , True),
                            StructField("total_amount"            , DoubleType()    , True),
                            StructField("congestion_surcharge"    , DoubleType()    , True),
                        ])
                    )
```

Spark Tables

And load:

```
yellowTaxiDF = (
                  spark
                    .read
                    .option("header", "true")
                    .schema(yellowTaxiSchema)
                    .csv("yellow_tripdata_2020-04.csv")
                )
yellowTaxiDF.printSchema()
```

Create a metastore with Hive

First let's show the existing and default databases:

```
spark.sql("""SHOW DATABASES""").show()
```

And create a new one for our taxis:

```
spark.sql("""CREATE DATABASE IF NOT EXISTS TaxisDB""")
spark.sql("""SHOW DATABASES""").show()
```

Managed Table

Save the Taxi data as a table

```
(
    yellowTaxiDF
            .write
            .mode("overwrite")
            .saveAsTable("TaxisDB.YellowTaxisManaged")
)
```

You can take a look at the catalog, if you still have the previous labs in memory, they should show up as `isTemporary` =true

```
spark.sql("""SHOW TABLES IN TaxisDB""").show(20, truncate=False)
```

Managed Table

Let's run a simple query

```
spark.sql("""
SELECT *
FROM TaxisDB.YellowTaxisManaged
LIMIT 10
""").show()
```

You can use the command spark.read as well for the same result:

```
outputDF = (
                spark
                    .read
                    .table("TaxisDB.YellowTaxisManaged")
            )

outputDF.limit(10).show()
```

Managed Table

Let's check the details of our YellowTaxisManaged table:

```
spark.sql("""DESCRIBE TABLE EXTENDED TaxisDB.YellowTaxisManaged""").show(50, truncate=False)
```

```
+----------------------------+-----------------------------------------------------------+-------+
|col_name                    |data_type                                                  |comment|
+----------------------------+-----------------------------------------------------------+-------+
|VendorId                    |int                                                        |null   |
|lpep_pickup_datetime        |timestamp                                                  |null   |
|lpep_dropoff_datetime       |timestamp                                                  |null   |
|passenger_count             |double                                                     |null   |
|trip_distance               |double                                                     |null   |
|RatecodeID                  |double                                                     |null   |
|store_and_fwd_flag          |string                                                     |null   |
|PULocationID                |int                                                        |null   |
|DOLocationID                |int                                                        |null   |
|payment_type                |int                                                        |null   |
|fare_amount                 |double                                                     |null   |
|extra                       |double                                                     |null   |
|mta_tax                     |double                                                     |null   |
|tip_amount                  |double                                                     |null   |
|tolls_amount                |double                                                     |null   |
|improvement_surcharge       |double                                                     |null   |
|total_amount                |double                                                     |null   |
|congestion_surcharge        |double                                                     |null   |
|airport_fee                 |double                                                     |null   |
|                            |                                                           |       |
|# Detailed Table Information|                                                           |       |
|Catalog                     |spark_catalog                                              |       |
|Database                    |taxisdb                                                    |       |
|Table                       |yellowtaxismanaged                                         |       |
|Created Time                |Sat Sep 16 12:59:02 UTC 2023                               |       |
|Last Access                 |UNKNOWN                                                    |       |
|Created By                  |Spark 3.4.1                                                |       |
|Type                        |MANAGED                                                    |       |
|Provider                    |parquet                                                    |       |
|Location                    |file:/content/spark-warehouse/taxisdb.db/yellowtaxismanaged|       |
+----------------------------+-----------------------------------------------------------+-------+
```

Managed Table

Please note that because this is a managed table, if you delete the table the data will be deleted as well.

External / Unmanaged table

The only difference in the command is that this time you precise the path option for the file.

```
(
    yellowTaxiDF
            .write
            .mode("overwrite")
            .option("path", "YellowTaxisOutput.parquet")
            .option("format", "parquet")       # Default is 'parquet'
            .saveAsTable("TaxisDB.YellowTaxis")
)
```

External / Unmanaged table

You can see the file in your local folder:

```
!ls
```

In addition to being listed in the metastore:

```
spark.sql("""
DESCRIBE TABLE EXTENDED TaxisDB.YellowTaxis
""").show(50, truncate=False)
```

col_name	data_type	comment
VendorId	int	null
lpep_pickup_datetime	timestamp	null
lpep_dropoff_datetime	timestamp	null
passenger_count	double	null
trip_distance	double	null
RatecodeID	double	null
store_and_fwd_flag	string	null
PULocationID	int	null
DOLocationID	int	null
payment_type	int	null
fare_amount	double	null
extra	double	null
mta_tax	double	null
tip_amount	double	null
tolls_amount	double	null
improvement_surcharge	double	null
total_amount	double	null
congestion_surcharge	double	null
airport_fee	double	null
# Detailed Table Information		
Catalog	spark_catalog	
Database	taxisdb	
Table	yellowtaxis	
Created Time	Sat Sep 16 13:11:42 UTC 2023	
Last Access	UNKNOWN	
Created By	Spark 3.4.1	
Type	EXTERNAL	
Provider	parquet	
Location	file:/content/spark-warehouse/taxisdb.db/YellowTaxisOutput.parquet	
Storage Properties	[format=parquet]	

External / Unmanaged table

Let's drop the external table and see what happened:

```
spark.sql("""DROP TABLE TaxisDB.YellowTaxis""")
```

```
spark.sql(""
SHOW TABLES IN TaxisDB
"").show(50, truncate=False)
```

The table is dropped but as you can see the data has not moved as expected

```
!ls
```

External / Unmanaged table

You can directly declare the table again directly in SQL using the "USING PARQUET" method:

```
spark.sql("""
CREATE TABLE TaxisDB.YellowTaxis
USING PARQUET
LOCATION "YellowTaxisOutput.parquet"
""")
```

Note: Parquet is a file format that contains its own schema so you do not need to declare one

```
spark.sql("""
SHOW TABLES IN TaxisDB
""").show(50, truncate=False)
```